Project Title:
Real-Time AI Assistant (RAG Chatbot)

Developed By:
Chinmaya Sajeevan

Technology:
Artificial Intelligence | NLP | Retrieval Augmented Generation (RAG)

Tools Used:
Python, Streamlit, LangChain, Ollama, DuckDuckGo Search

Project Type:
AI Chatbot Application

Real-Time AI Assistant using Ollama + LangChain + DuckDuckGo

Introduction

Artificial Intelligence chatbots normally rely only on pre-trained knowledge, which may become outdated. This project implements a **Real-Time AI Assistant** capable of answering user questions using live web search results.

The chatbot combines a **Large Language Model (LLM)** with **real-time information retrieval**, allowing it to generate accurate and up-to-date responses.

The application provides a conversational interface where users can ask questions and receive answers generated from current internet data.

Problem Statement

Traditional chatbots suffer from:

- Limited knowledge cutoff dates
- Lack of real-time information
- Static responses without external verification

The objective of this project is to build an **AI assistant that retrieves live information from the web and generates context-aware answers** using Retrieval Augmented Generation (RAG)

Objectives

- Build an AI chatbot using Ollama LLM
- Retrieve live information using DuckDuckGo Search
- Integrate LangChain for workflow orchestration
- Create an interactive chat interface using Streamlit
- Provide real-time intelligent responses

Technologies & Libraries Used

Technology	Purpose
Python	Programming Language
Streamlit	Web Interface
Ollama	Local LLM execution
LangChain	AI pipeline orchestration
DuckDuckGo Search	Real-time information retrieval
Llama3 Model	Language understanding & generation

System Workflow

1. User enters a question.
2. DuckDuckGo searches the web for relevant information.
3. Search results are passed as context.
4. LangChain builds the prompt dynamically.
5. Ollama Llama3 model generates the response.
6. Answer is displayed in a chat interface.

Project Architecture

User Query
↓
DuckDuckGo Search
↓
LangChain Prompt Template
↓
Ollama LLM (Llama3)
↓
Generated Response
↓
Streamlit Chat UI

Code Snippets

```
import streamlit as st

from langchain_ollama import OllamaLLM
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
```

```
#Page config
st.set_page_config(
    page_title="Ollama RAG Chatbot",
    layout="centered")

st.title("Real-time AI Assistant")
st.caption("Powered by Ollama + DuckDuckGo + Langchain")
```

```
#Initialize LLM & tools
llm=OllamaLLM(model="llama3:8b")
search=DuckDuckGoSearchRun()
```

```
prompt=ChatPromptTemplate.from_template(
    """
    You are a helpful AI assistant. You must answer the user's question
    based *only* on the following search results.
    If the search results are empty or do not contain the answer,
    say "I could not find any information on that."

    Search Results:
    {context}

    Question:
    {question}
    """
)
```

```
#Chain creation core logic
chain=(
    RunnablePassthrough.assign(
        context=lambda x: search.run(x["question"])
    )
    | prompt
    | llm
)
```

```python
if "messages" not in st.session_state:
    st.session_state.messages=[]

#Display previous messsages
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
```

```python
#user input
user_query=st.chat_input("ask me anything...")

if user_query:
    #show user message
    st.session_state.messages.append(
        {"role": "user","content": user_query}
    )
    with st.chat_message("user"):
        st.markdown(user_query)

    #Generate response
    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            response=chain.invoke({"question": user_query})
            st.markdown(response)

    #Save assistant message
    st.session_state.messages.append(
        {"role": "assistant", "content": response}
    )
```

How to Run the Project

Step 1 — Install Requirements

pip install streamlit langchain langchain-community langchain-ollama duckduckgo-search

Step 2 — Install Ollama & Model

ollama run llama3:8b

Step 3 — Run Application

streamlit run assistant.py

Features

- Real-time web search integration
- Local LLM execution (no API key required)
- Conversational chat interface
- Context-aware responses
- Retrieval Augmented Generation (RAG)

Results

The chatbot successfully retrieves live information and generates accurate responses based only on retrieved search results, improving reliability compared to static chatbots.

Future Enhancements

- Add memory for long conversations
- Voice interaction support
- Multiple LLM selection
- Deployment on cloud platforms

Conclusion

This project demonstrates how Retrieval Augmented Generation (RAG) can enhance AI chatbot performance by combining real-time search with Large Language Models. The system provides dynamic, up-to-date answers while maintaining an intuitive conversational interface.